Lab Flutter: Thanh toán qua Cổng REST API (Mock Gateway)

Tài liệu hướng dẫn thực hành từng bước. Thời lượng dự kiến: 90–120 phút.

1. Mục tiêu bài lab

Sau bài lab, bạn sẽ:

- Hiểu luồng thanh toán chuẩn qua cổng REST API: **tạo giao dịch → nhận mã giao dịch → chờ xác nhận → hiển thị kết quả**.
- Gửi HTTP POST có **chữ ký HMAC-SHA256** từ Flutter.
- Dùng **polling** để theo dõi trạng thái giao dịch (PENDING / SUCCESS / FAILED).
- Xử lý lỗi mạng, timeout, trạng thái loading đúng cách.
- Biết cách thay Mock Gateway bằng cổng thật (PayOS) ở phần mở rộng.

2. Vì sao chọn Mock Gateway?

Với bài lab cho nhiều người, Mock Gateway có lợi thế rõ rệt so với hai lựa chọn còn lại:

- **Ví MoMo / ZaloPay SDK:** cần đăng ký merchant, cần thiết bị thật cài app, cấu hình native Android/iOS phức tạp, dễ lỗi môi trường.
- **VietQR động:** sinh mã QR dễ, nhưng muốn biết khách đã chuyển tiền chưa thì phải có backend nhận webhook từ ngân hàng.
- **REST API + Mock Gateway:** không cần tài khoản doanh nghiệp, chạy được trên emulator, ai cũng làm được. Luồng gọi API và xử lý trạng thái giống hệt cổng thật.

3. Yêu cầu chuẩn bị

- Flutter SDK 3.19 trở lên (kèm Dart 3.x), đã chạy được `flutter doctor`.
- Một emulator/simulator hoặc thiết bị thật.
- Terminal để chạy server giả lập và lệnh `curl`.
- Kiến thức cơ bản: widget, StatefulWidget, async/await.

4. Luồng hoạt động

1. App gửi `POST /api/payments` kèm `orderId`, `amount`, `signature`.
2. Gateway kiểm tra chữ ký, tạo giao dịch trạng thái `PENDING`, trả về `id`.

3. App chuyển sang màn hình chờ, cứ 2 giây gọi `GET /api/payments/{id}` một lần.
4. Khách "thanh toán" (trong lab này bạn giả lập bằng lệnh `curl`) → Gateway đổi trạng thái thành `SUCCESS` hoặc `FAILED`.
5. App nhận trạng thái mới và hiển thị kết quả.

5. Cấu trúc thư mục

```
payment_lab/
├── server/                 # Mock Gateway (Dart)
|   ├── pubspec.yaml
|   └── bin/server.dart
└── app/                    # Ứng dụng Flutter
    ├── pubspec.yaml
    └── lib/
        ├── main.dart
        ├── models/payment.dart
        ├── services/payment_api.dart
        └── screens/
            ├── checkout_screen.dart
            └── payment_status_screen.dart
```

6. Phần A: Dựng Mock Gateway

Bước A1. Tạo project server

```
mkdir payment_lab && cd payment_lab
dart create -t console server
cd server
dart pub add shelf shelf_router crypto
```

Bước A2. Viết server

Thay toàn bộ nội dung `server/bin/server.dart` bằng:

```
import 'dart:convert';
import 'dart:io';

import 'package:crypto/crypto.dart';
import 'package:shelf/shelf.dart';
import 'package:shelf/shelf_io.dart' as io;
import 'package:shelf_router/shelf_router.dart';
```

```
const secretKey = 'lab-secret-key';
final payments = <String, Map<String, dynamic>>{};
int _counter = 1000;

String sign(String data) =>
    Hmac(sha256,
utf8.encode(secretKey)).convert(utf8.encode(data)).toString();

Response jsonResponse(Object body, {int status = 200}) => Response(
      status,
      body: jsonEncode(body),
      headers: {'content-type': 'application/json'},
    );

void main() async {
  final router = Router();

  // 1. Tạo giao dịch
  router.post('/api/payments', (Request req) async {
    final body = jsonDecode(await req.readAsString()) as Map<String,
dynamic>;
    final amount = body['amount'];
    final orderId = body['orderId'];
    final signature = body['signature'];

    if (amount is! int || amount <= 0 || orderId is! String) {
      return jsonResponse(
        {'code': 'INVALID_DATA', 'message': 'Dữ liệu không hợp lệ'},
        status: 400,
      );
    }
    if (signature != sign('amount=$amount&orderId=$orderId')) {
      return jsonResponse(
        {'code': 'INVALID_SIGNATURE', 'message': 'Chữ ký không hợp lệ'},
        status: 401,
      );
    }

    final id = 'PAY${_counter++}';
    payments[id] = {
      'id': id,
      'orderId': orderId,
      'amount': amount,
      'status': 'PENDING',
      'createdAt': DateTime.now().toIso8601String(),
    };
```

```
    return jsonResponse(payments[id]!, status: 201);
  });

  // 2. Xem trạng thái
  router.get('/api/payments/<id>', (Request req, String id) {
    final p = payments[id];
    if (p == null) {
      return jsonResponse(
        {'code': 'NOT_FOUND', 'message': 'Không tìm thấy giao dịch'},
        status: 404,
      );
    }
    return jsonResponse(p);
  });

  // 3. Giả lập khách thanh toán (chỉ dùng trong lab)
  router.post('/api/payments/<id>/confirm', (Request req, String id) async {
    final p = payments[id];
    if (p == null) {
      return jsonResponse({'code': 'NOT_FOUND', 'message': 'Không tìm thấy'},
status: 404);
    }
    final body = jsonDecode(await req.readAsString()) as Map<String,
dynamic>;
    p['status'] = body['result'] == 'failed' ? 'FAILED' : 'SUCCESS';
    return jsonResponse(p);
  });

  final handler =
      const Pipeline().addMiddleware(logRequests()).addHandler(router.call);
  final server = await io.serve(handler, InternetAddress.anyIPv4, 8080);
  print('Mock Gateway chạy tại http://localhost:${server.port}');
}
```

Bước A3. Chạy và thử server

```
dart run bin/server.dart
```

Mở terminal khác, kiểm tra route kiểm tra trạng thái (chắc chắn trả về 404 vì chưa có giao dịch):

```
curl -i http://localhost:8080/api/payments/PAY1000
```

Thấy `404` và JSON `NOT_FOUND` là server đã chạy đúng. Để server chạy nền trong suốt bài lab.

7. Phần B: Xây dựng app Flutter

Bước B1. Tạo project và thêm thư viện

```
cd ..   # quay về payment_lab
flutter create app
cd app
flutter pub add http crypto
```

Bước B2. Cho phép HTTP thường trên Android

Mock server dùng `http://` (không phải https). Mở `android/app/src/main/AndroidManifest.xml`, thêm thuộc tính vào thẻ `<application>`:

```
<application
    android:usesCleartextTraffic="true"
    ... >
```

Với iOS simulator không cần cấu hình thêm khi chạy `localhost`.

Bước B3. Model: `lib/models/payment.dart`

```
enum PaymentStatus { pending, success, failed }

class Payment {
  final String id;
  final String orderId;
  final int amount;
  final PaymentStatus status;

  const Payment({
    required this.id,
    required this.orderId,
    required this.amount,
    required this.status,
  });

  factory Payment.fromJson(Map<String, dynamic> json) => Payment(
        id: json['id'] as String,
        orderId: json['orderId'] as String,
        amount: json['amount'] as int,
        status: switch (json['status']) {
          'SUCCESS' => PaymentStatus.success,
          'FAILED' => PaymentStatus.failed,
```

```
          _ => PaymentStatus.pending,
        },
      );
}
```

Bước B4. Service gọi API: `lib/services/payment_api.dart`

```
import 'dart:convert';
import 'dart:io';

import 'package:crypto/crypto.dart';
import 'package:flutter/foundation.dart';
import 'package:http/http.dart' as http;

import '../models/payment.dart';

class PaymentException implements Exception {
  final String message;
  PaymentException(this.message);
  @override
  String toString() => message;
}

class PaymentApi {
  PaymentApi({http.Client? client}) : _client = client ?? http.Client();
  final http.Client _client;

  // Android emulator gọi máy host qua 10.0.2.2.
  // Thiết bị thật: thay bằng IP LAN của máy chạy server.
  static String get baseUrl {
    if (!kIsWeb && Platform.isAndroid) return 'http://10.0.2.2:8080';
    return 'http://localhost:8080';
  }

  // Lưu ý: khóa bí mật KHÔNG được để trong app ở sản phẩm thật.
  // Ở đây để trong app chỉ nhằm mục đích học.
  static const _secretKey = 'lab-secret-key';

  String _sign(String data) => Hmac(sha256, utf8.encode(_secretKey))
      .convert(utf8.encode(data))
      .toString();

  Future<Payment> createPayment({
    required String orderId,
    required int amount,
  }) async {
```

```
    try {
      final res = await _client
          .post(
            Uri.parse('$baseUrl/api/payments'),
            headers: {'Content-Type': 'application/json'},
            body: jsonEncode({
              'orderId': orderId,
              'amount': amount,
              'signature': _sign('amount=$amount&orderId=$orderId'),
            }),
          )
          .timeout(const Duration(seconds: 10));

      if (res.statusCode != 201) throw PaymentException(_errorOf(res));
      return Payment.fromJson(jsonDecode(res.body));
    } on SocketException {
      throw PaymentException('Không kết nối được tới cổng thanh toán');
    } on http.ClientException {
      throw PaymentException('Không kết nối được tới cổng thanh toán');
    }
  }

  Future<Payment> getPayment(String id) async {
    final res = await _client
        .get(Uri.parse('$baseUrl/api/payments/$id'))
        .timeout(const Duration(seconds: 10));
    if (res.statusCode != 200) throw PaymentException(_errorOf(res));
    return Payment.fromJson(jsonDecode(res.body));
  }

  String _errorOf(http.Response res) {
    try {
      return (jsonDecode(res.body) as Map)['message'] as String;
    } catch (_) {
      return 'Lỗi không xác định (${res.statusCode})';
    }
  }
}
```

Bước B5. Màn hình thanh toán: `lib/screens/checkout_screen.dart`

```
import 'package:flutter/material.dart';

import '../models/payment.dart';
import '../services/payment_api.dart';
import 'payment_status_screen.dart';
```

```
String formatVnd(int v) =>
    '${v.toString().replaceAllMapped(RegExp(r'\B(?=(\d{3})+(?!\d))'), (_) =>
'.')} đ';

class CheckoutScreen extends StatefulWidget {
  const CheckoutScreen({super.key});
  @override
  State<CheckoutScreen> createState() => _CheckoutScreenState();
}

class _CheckoutScreenState extends State<CheckoutScreen> {
  final _api = PaymentApi();
  final int _amount = 150000;
  bool _loading = false;

  Future<void> _pay() async {
    setState(() => _loading = true);
    try {
      final orderId = 'ORDER${DateTime.now().millisecondsSinceEpoch}';
      final Payment payment =
          await _api.createPayment(orderId: orderId, amount: _amount);
      if (!mounted) return;
      Navigator.of(context).push(MaterialPageRoute(
        builder: (_) => PaymentStatusScreen(api: _api, payment: payment),
      ));
    } on PaymentException catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context)
          .showSnackBar(SnackBar(content: Text(e.message)));
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Thanh toán đơn hàng')),
      body: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            const ListTile(
              leading: Icon(Icons.shopping_bag_outlined),
```

```
              title: Text('Khóa học Flutter cơ bản'),
              subtitle: Text('Số lượng: 1'),
            ),
            const Divider(),
            ListTile(
              title: const Text('Tổng tiền'),
              trailing: Text(formatVnd(_amount),
                  style: Theme.of(context).textTheme.titleLarge),
            ),
            const Spacer(),
            FilledButton(
              onPressed: _loading ? null : _pay,
              child: _loading
                  ? const SizedBox(
                      height: 20,
                      width: 20,
                      child: CircularProgressIndicator(strokeWidth: 2))
                  : const Text('Thanh toán'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Bước B6. Màn hình chờ kết quả: `lib/screens/payment_status_screen.dart`

```
import 'dart:async';

import 'package:flutter/material.dart';

import '../models/payment.dart';
import '../services/payment_api.dart';
import 'checkout_screen.dart' show formatVnd;

class PaymentStatusScreen extends StatefulWidget {
  const PaymentStatusScreen({super.key, required this.api, required
this.payment});
  final PaymentApi api;
  final Payment payment;

  @override
  State<PaymentStatusScreen> createState() => _PaymentStatusScreenState();
}
```

```
class _PaymentStatusScreenState extends State<PaymentStatusScreen> {
  late Payment _payment = widget.payment;
  Timer? _timer;
  int _elapsed = 0;
  bool _timedOut = false;

  static const _interval = 2; // giây
  static const _maxWait = 60; // giây

  @override
  void initState() {
    super.initState();
    _timer = Timer.periodic(const Duration(seconds: _interval), (_) =>
_poll());
  }

  Future<void> _poll() async {
    _elapsed += _interval;
    if (_elapsed >= _maxWait) {
      _timer?.cancel();
      if (mounted) setState(() => _timedOut = true);
      return;
    }
    try {
      final p = await widget.api.getPayment(_payment.id);
      if (!mounted) return;
      setState(() => _payment = p);
      if (p.status != PaymentStatus.pending) _timer?.cancel();
    } catch (_) {
      // Lỗi mạng tạm thời: bỏ qua, lần poll sau sẽ thử lại.
    }
  }

  @override
  void dispose() {
    _timer?.cancel(); // QUAN TRỌNG: tránh rò rỉ Timer
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final (icon, color, text) = switch (_payment.status) {
      PaymentStatus.success => (Icons.check_circle, Colors.green, 'Thanh toán
thành công'),
      PaymentStatus.failed => (Icons.cancel, Colors.red, 'Thanh toán thất
bại'),
```

```
      PaymentStatus.pending => _timedOut
          ? (Icons.timer_off, Colors.orange, 'Hết thời gian chờ')
          : (Icons.hourglass_top, Colors.blue, 'Đang chờ thanh toán...'),
    };

    return Scaffold(
      appBar: AppBar(title: const Text('Kết quả giao dịch')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Icon(icon, size: 80, color: color),
            const SizedBox(height: 16),
            Text(text, style: Theme.of(context).textTheme.titleLarge),
            const SizedBox(height: 8),
            Text('Mã giao dịch: ${_payment.id}'),
            Text('Số tiền: ${formatVnd(_payment.amount)}'),
            const SizedBox(height: 24),
            if (_payment.status != PaymentStatus.pending || _timedOut)
              OutlinedButton(
                onPressed: () => Navigator.of(context).pop(),
                child: const Text('Quay lại'),
              ),
          ],
        ),
      ),
    );
  }
}
```

Bước B7. `lib/main.dart`

```
import 'package:flutter/material.dart';

import 'screens/checkout_screen.dart';

void main() => runApp(const PaymentLabApp());

class PaymentLabApp extends StatelessWidget {
  const PaymentLabApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Payment Lab',
      theme: ThemeData(colorSchemeSeed: Colors.indigo, useMaterial3: true),
      home: const CheckoutScreen(),
    );
  }
}
```

8. Chạy thử toàn bộ luồng

1. Đảm bảo server đang chạy (Bước A3).
2. Chạy app: `flutter run`.
3. Bấm **Thanh toán**. App chuyển sang màn hình "Đang chờ thanh toán..." và hiện mã giao dịch, ví dụ `PAY1000`.
4. Ở terminal khác, giả lập khách thanh toán thành công:

```
curl -X POST http://localhost:8080/api/payments/PAY1000/confirm \
  -H "Content-Type: application/json" \
  -d '{"result":"success"}'
```

5. Trong vòng 2 giây, app chuyển sang "Thanh toán thành công".
6. Làm lại và thay `success` bằng `failed` để thấy trạng thái thất bại.

9. Danh sách kiểm tra (checklist)

- [ ] Bấm thanh toán nhận được mã giao dịch từ server.
- [ ] Nút thanh toán bị khóa và hiện vòng quay khi đang gọi API.
- [ ] Tắt server rồi bấm thanh toán: app hiện thông báo lỗi kết nối, không crash.
- [ ] Xác nhận `success` → màn hình xanh. Xác nhận `failed` → màn hình đỏ.

- [ ] Không xác nhận gì trong 60 giây → hiện "Hết thời gian chờ".
- [ ] Thoát màn hình chờ giữa chừng: không còn log poll chạy ngầm trên server.

10. Bài tập tự làm

Bài 1 (dễ): Cho phép người dùng nhập số tiền thay vì cố định 150.000 đ. Kiểm tra số tiền phải lớn hơn 0.

Bài 2 (dễ): Trên màn hình chờ, thêm nút **Hủy giao dịch**. Viết thêm endpoint `POST /api/payments/<id>/cancel` ở server, trạng thái mới là `CANCELLED`.

Bài 3 (vừa): Viết unit test cho `PaymentApi` bằng `MockClient` của package `http/testing.dart`, kiểm tra 3 trường hợp: thành công (201), sai chữ ký (401), lỗi mạng.

Bài 4 (vừa): Thêm màn hình **Lịch sử giao dịch**. Server bổ sung `GET /api/payments` trả danh sách, app hiển thị bằng `ListView`.

Bài 5 (khó): Thêm cơ chế **idempotency**: nếu app gửi lại cùng một `orderId` (do người dùng bấm hai lần), server trả về giao dịch cũ thay vì tạo giao dịch mới.

Bài 6 (khó): Thay polling bằng WebSocket hoặc Server-Sent Events để server chủ động báo trạng thái.

11. Lỗi thường gặp

- **`Connection refused` trên Android emulator:** bạn đang dùng `localhost`. Emulator phải dùng `10.0.2.2`.
- **`Cleartext HTTP traffic not permitted`:** quên thêm `usesCleartextTraffic` ở Bước B2.
- **Thiết bị thật không kết nối được:** điện thoại và máy tính phải cùng Wi-Fi, dùng IP LAN của máy (ví dụ `192.168.1.10`), và tường lửa cho phép cổng 8080.
- **`INVALID_SIGNATURE`:** chuỗi ký phải đúng thứ tự `amount=...&orderId=...` ở cả server và app, và dùng cùng khóa bí mật.
- **Cảnh báo `setState() called after dispose()`:** thiếu `mounted` check hoặc quên `_timer?.cancel()` trong `dispose`.

12. Lưu ý bảo mật (rất quan trọng)

Bài lab đặt khóa bí mật và ký chữ ký ngay trong app để dễ hiểu. **Trong sản phẩm thật điều này là sai**, vì ai cũng có thể giải mã app để lấy khóa. Cách đúng:

- App chỉ gọi **backend của bạn**. Backend giữ khóa bí mật và gọi cổng thanh toán.

- Trạng thái giao dịch phải được xác nhận bằng **webhook** từ cổng gửi về backend, không tin kết quả do app tự báo.
- Luôn dùng **HTTPS**.

13. Mở rộng: chuyển sang cổng PayOS thật

Khi đã nắm vững luồng trên, bạn có thể thay Mock Gateway bằng PayOS:

1. Đăng ký tài khoản và tạo kênh thanh toán trên PayOS để lấy `Client ID`, `API Key`, `Checksum Key`.
2. Dựng một backend nhỏ (Dart shelf, Node.js...) gọi API tạo link thanh toán của PayOS, giữ các khóa ở backend.
3. App Flutter gọi backend của bạn để lấy `checkoutUrl` hoặc mã QR, mở bằng `url_launcher` hoặc hiển thị QR.
4. Backend nhận webhook từ PayOS, cập nhật trạng thái; app tiếp tục polling `GET /api/payments/{id}` như trong lab này.

Vì app chỉ phụ thuộc vào `PaymentApi`, phần lớn code giao diện giữ nguyên. Đó là lý do bài lab tách riêng lớp service.

Chi tiết tham số và endpoint hiện hành, hãy xem tài liệu chính thức của PayOS trước khi triển khai vì có thể đã thay đổi.